4.1 Cumulus 구조 및 파라체인 템플릿 분석

개요

Cumulus는 Substrate 기반의 Polkadot 파라체인을 개발하기 위한 SDK입니다. "Cumulus clouds are shaped sort of like dots; together they form a system that is intricate, beautiful and functional"이라는 설명처럼, Cumulus는 개별 파라체인들이 하나의 통합된 Polkadot 생태계를 형성할 수 있도록 돕는 핵심 프레임워크입니다.

Cumulus는 Substrate의 블록체인 개발 편의성을 파라체인 개발에 그대로 적용하면서, 릴레이 체인과의 통신, 합의 메커니즘, 크로스체인 메시징 등 파라체인 특화 기능을 제공합니다.

Cumulus 디렉토리 구조 분석

4.1.1 최상위 디렉토리 구조

```
cumulus/
├── bin/ # 실행 가능한 바이너리들
├── client/ # 클라이언트 구현체들
├── docker/ # Docker 관련 설정
├── docs/ # 문서
├── pallets/ # Cumulus 전용 팔렛들
├── parachains/ # 실제 파라체인 구현체들
├── polkadot-omni-node/ # 범용 파라체인 노드
├── polkadot-parachain/ # 파라체인 바이너리
```

```
├── primitives/ # 핵심 프리미티브들
├── scripts/ # 유틸리티 스크립트들
├── templates/ # 템플릿 파일들
├── test/ # 테스트 코드
├── xcm/ # XCM 관련 구현
└── zombienet/ # 네트워크 테스트 설정
```

4.1.2 핵심 컴포넌트 상세 분석

Client 디렉토리 (`cumulus/client/`)

클라이언트 디렉토리는 파라체인 노드의 핵심 기능을 구현합니다:

```
client/
├── bootnodes/ # 부트노드 관리
├── cli/ # 명령줄 인터페이스
├── collator/ # 콜레이터 구현
├── consensus/ # 합의 메커니즘
├── network/ # 네트워킹 레이어
├── parachain-inherent/ # 파라체인 고유 데이터 처리
├── pov-recovery/ # PoV(Proof of Validity) 복구
├── relay-chain-interface/ # 릴레이 체인 인터페이스
├── relay-chain-inprocess-interface/ # 인프로세스 릴레이 체인
├── relay-chain-minimal-node/ # 최소 릴레이 체인 노드
├── relay-chain-rpc-interface/ # RPC 기반 릴레이 체인 연결
```

```
├── relay-chain-streams/ # 릴레이 체인 스트림
└── service/ # 노드 서비스 구현
```

주요 특징:

- **콜레이터 시스템**: 파라체인 블록을 생성하고 릴레이 체인에 제출하는 역할
- **릴레이 체인 연결**: 인프로세스 또는 RPC를 통한 릴레이 체인 연결 지원
- **합의 메커니즘**: 릴레이 체인을 따라가는 파라체인 전용 합의 구현

Primitives 디렉토리 (`cumulus/primitives/`)

파라체인 개발에 필요한 핵심 데이터 타입과 트레이트들을 정의합니다:

```
primitives/
├── aura/ # Aura 합의 관련 프리미티브
├── core/ # 핵심 데이터 타입
├── parachain-inherent/ # 파라체인 고유 데이터
├── proof-size-hostfunction/ # 증명 크기 관련 호스트 함수
├── storage-weight-reclaim/ # 스토리지 가중치 회수
├── timestamp/ # 타임스탬프 관련
└── utility/ # 유틸리티 함수들
```

Pallets 디렉토리 (`cumulus/pallets/`)

Cumulus 전용 팔렛들로 파라체인 특화 기능을 제공합니다:

```
pallets/
├── aura-ext/ # Aura 확장 팔렛
```

```
├── collator-selection/ # 콜레이터 선택 팔렛
├── dmp-queue/ # DMP(Downward Message Passing) 큐
├── parachain-system/ # 파라체인 시스템 팔렛 (핵심)
├── session-benchmarking/ # 세션 벤치마킹
├── solo-to-para/ # 솔로체인에서 파라체인으로 전환
├── weight-reclaim/ # 가중치 회수 팔렛
├── xcm/ # XCM 팔렛
└── xcmp-queue/ # XCMP(Cross-Chain Message Passing) 큐
```

핵심 팔렛 - parachain-system:

```
//! `cumulus-pallet-parachain-system`은 Cumulus 기반 파라체인의 기본 팔렛입니다.
//!
//! 이 팔렛의 주요 책임:
//! - 파라체인 검증 데이터 수집
//! - 다운워드 및 래터럴 메시지 수집 및 디스패치
//! - 릴레이 체인과의 업그레이드 조정
//! - 전송된 메시지, 업그레이드 신호 등 파라체인 출력 통신
```

파라체인 템플릿 구조 분석

4.1.3 파라체인 템플릿 개요

Polkadot SDK는 `templates/parachain/` 디렉토리에 완전한 파라체인 템플릿을 제공합니다. 이 템플릿은 개발자가 빠르게 파라체인 개발을 시작할 수 있도록 설계되었습니다.

템플릿 구조

```
templates/parachain/
├── node/ # 노드 구현
| ├── src/
| | ├── chain_spec.rs # 체인 스펙 정의
| | ├── service.rs # 노드 서비스 구현
| | └── main.rs # 메인 엔트리포인트
| ├── build.rs
| └── Cargo.toml
├── runtime/ # 런타임 구현
| ├── src/
| | └── lib.rs # 런타임 정의
| ├── build.rs
| └── Cargo.toml
├── pallets/ # 커스텀 팔렛들
| └── template/ # 템플릿 팔렛
└── src/
└── lib.rs # 라이브러리 루트
```

4.1.4 주요 컴포넌트 상세 분석

1. 런타임 (Runtime)

파라체인 런타임은 Substrate FRAME을 기반으로 하되, Cumulus 특화 기능이 추가됩니다:

핵심 특징:

- **FRAME 기반**: 기존 Substrate 팔렛들을 재사용
- **파라체인 시스템 팔렛**: `cumulus_pallet_parachain_system` 필수 포함
- **XCM 지원**: 크로스체인 메시징을 위한 XCM 팔렛들 포함
- **검증 블록 등록**: `cumulus_pallet_parachain_system::register_validate_block!` 매크로 사용

런타임 구성 예시:

```
// 파라체인 ID 정의
pub const PARACHAIN_ID: u32 = 1000;


// 파라체인 검증 블록 등록
cumulus_pallet_parachain_system::register_validate_block!(Block, Executive);


// 런타임 구성
construct_runtime!(
pub struct Runtime {
// 기본 시스템 팔렛들
System: frame_system,
Timestamp: pallet_timestamp,
Balances: pallet_balances,
// 파라체인 전용 팔렛들
ParachainSystem: cumulus_pallet_parachain_system,
ParachainInfo: parachain_info,
```

```
// XCM 관련 팔렛들
XcmpQueue: cumulus_pallet_xcmp_queue,
PolkadotXcm: pallet_xcm,
// 커스텀 팔렛
TemplateModule: pallet_template,
}
);
```

2. 노드 (Node)

파라체인 노드는 다음과 같은 특징을 가집니다:

체인 스펙 (`chain_spec.rs`):

- 제네시스 상태 정의
- 파라체인 ID 설정
- 초기 계정 및 잔액 설정
- 콜레이터 설정

서비스 (`service.rs`):

- 파라체인 합의 구현
- 릴레이 체인 연결 설정
- 콜레이터 서비스 구성
- RPC 엔드포인트 설정

3. 팔렛 (Pallets)

템플릿에는 기본적인 커스텀 팔렛 예제가 포함되어 있습니다:

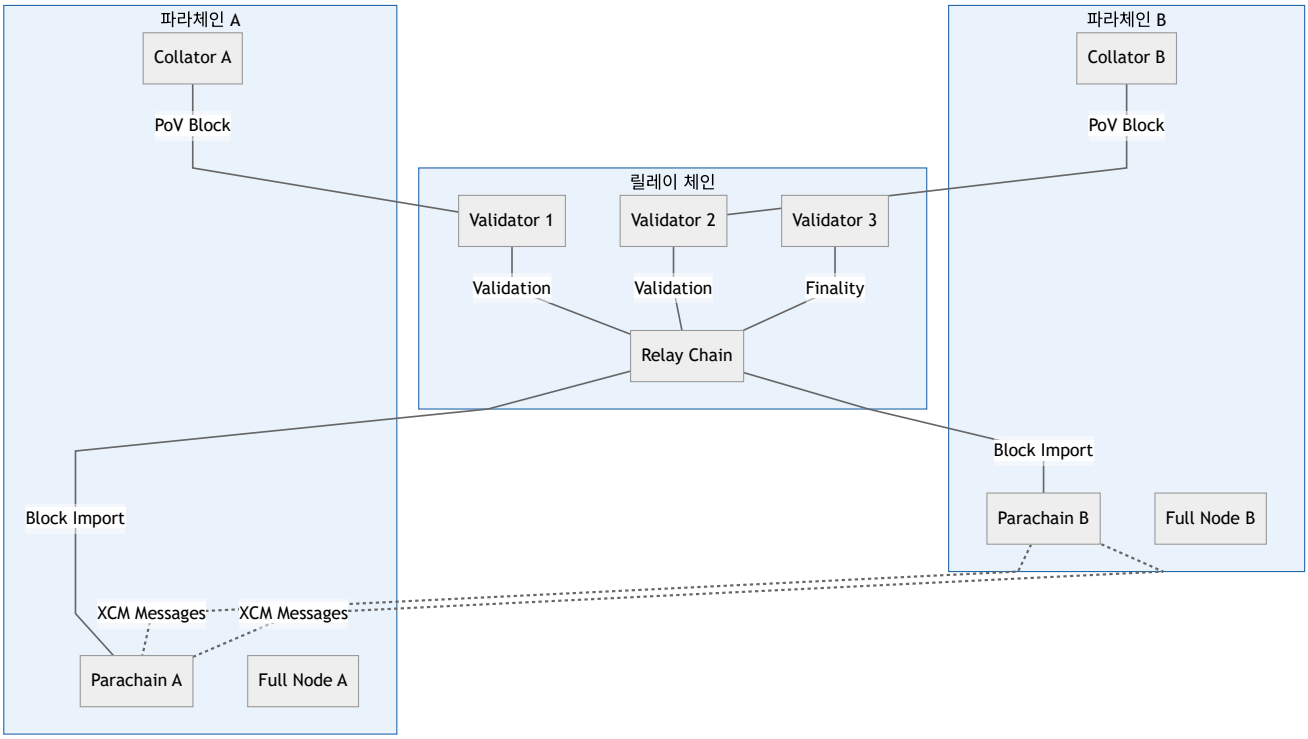
```
// 템플릿 팔렛 구조
#[pallet::pallet]
pub struct Pallet<T>(_);


#[pallet::config]
pub trait Config: frame_system::Config {
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
}


#[pallet::call]
impl<T: Config> Pallet<T> {
#[pallet::call_index(0)]
#[pallet::weight(Weight::from_parts(10_000, 0) + T::DbWeight::get().writes(1))]
pub fn do_something(origin: OriginFor<T>, something: u32) -> DispatchResult {
// 팔렛 로직 구현
Ok(())
}
}
```

파라체인과 릴레이 체인 간 통신 메커니즘

4.1.5 통신 아키텍처



4.1.6 핵심 통신 메커니즘

1. PoV (Proof of Validity) 블록

파라체인은 릴레이 체인에 PoV 블록을 제출합니다:

```
pub struct PoVBlock {
/// 파라체인 블록 헤더
pub block: Block,
```

```
/// 증명 데이터 (witness data)
pub witness_data: Vec<u8>,
/// 나가는 메시지들
pub outgoing_messages: Vec<OutboundMessage>,
}
```

특징:

- **증명 데이터**: 블록 실행에 필요한 스토리지 증명 포함
- **불투명한 형식**: 릴레이 체인은 파라체인 블록 형식을 알 필요 없음
- **검증 가능**: 전체 상태 없이도 블록 검증 가능

2. 합의 메커니즘

파라체인은 릴레이 체인을 따라가는 합의를 사용합니다:

```
// 파라체인 합의 구현
impl<Block: BlockT> Consensus<Block> for ParachainConsensus<Block> {
fn proposer(&mut self, block: &BlockId<Block>) -> Result<Self::Proposer, Error
// 릴레이 체인 상태를 기반으로 제안자 생성
}
fn finality_proof(&self, block: &BlockId<Block>) -> Result<Option<FinalityProof
// 릴레이 체인 최종성을 따라감
}
}
```

3. 메시지 전달 시스템

DMP (Downward Message Passing):

- 릴레이 체인 → 파라체인 메시지 전달
- 거버넌스 결정, 스테이킹 보상 등

UMP (Upward Message Passing):

- 파라체인 → 릴레이 체인 메시지 전달
- 스테이킹 요청, 거버넌스 제안 등

XCMP (Cross-Chain Message Passing):

- 파라체인 간 직접 메시지 전달
- 자산 전송, 원격 실행 등

파라체인 개발 환경 설정

4.1.7 개발 환경 준비

1. 기본 요구사항

```
# Rust 설치 (Windows)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup default stable
rustup update
rustup target add wasm32-unknown-unknown
```

```
# 추가 도구 설치
cargo install --git https://github.com/paritytech/polkadot-sdk polkadot-omni
```

```
cargo install --git https://github.com/paritytech/polkadot-sdk staging-chain
```

2. Polkadot SDK 클론 및 빌드

```
# 저장소 클론
git clone https://github.com/paritytech/polkadot-sdk
cd polkadot-sdk

# 파라체인 바이너리 빌드
cargo build --release -p polkadot-parachain-bin

# 릴레이 체인 바이너리 빌드 (테스트용)
cargo build --release -p polkadot
```

4.1.8 초기 프로젝트 생성 가이드

1. 템플릿 기반 프로젝트 생성

```
# 파라체인 템플릿 복사
cp -r templates/parachain my-parachain
cd my-parachain

# 프로젝트 설정 수정
# Cargo.toml에서 프로젝트 이름 변경
```

```
# runtime/src/lib.rs에서 PARACHAIN_ID 설정
```

2. 체인 스펙 생성

```
# 런타임 빌드
cargo build --profile production

# 체인 스펙 생성
chain-spec-builder create \
--relay-chain "rococo-local" \
--runtime target/release/wbuild/parachain-template-runtime/parachain_template_r
named-preset development
```

3. 개발 네트워크 실행

Omni Node 사용:

```
# 개발 모드로 실행
polkadot-omni-node \
--chain <path/to/chain_spec.json> \
--dev \
--dev-block-time 1000
```

Zombienet 사용:

```
# Zombienet 설치
npm install -g @zombienet/cli

# 네트워크 실행
zombienet --provider native spawn zombienet-omni-node.toml
```

4. 프로덕션 배포 준비

```
# 제네시스 상태 내보내기
./target/release/polkadot-parachain export-genesis-state > genesis-state

# 제네시스 WASM 내보내기
./target/release/polkadot-parachain export-genesis-wasm > genesis-wasm

# 콜레이터 실행
./target/release/polkadot-parachain \
--collator \
--alice \
--force-authoring \
--tmp \
--port 40335 \
--rpc-port 9946 \
-- \
```

```
--chain  rococo-local-cfde.json \

--port 30335
```

개발 도구 및 유틸리티

4.1.9 주요 개발 도구

1. Polkadot-JS Apps

- **용도**: 파라체인 상태 조회 및 트랜잭션 전송
- **접속**: https://polkadot.js.org/apps/
- **설정**: Custom endpoint로 로컬 노드 연결

2. Chopsticks

- **용도**: 런타임 개발 및 테스트
- **설치**: `npm install -g @acala-network/chopsticks`
- **사용**: 포크 테스팅 및 런타임 시뮬레이션

3. Zombienet

- **용도**: 멀티노드 네트워크 테스트
- **설치**: `npm install -g @zombienet/cli`
- **설정**: TOML 파일로 네트워크 구성

4.1.10 모니터링 및 디버깅

로그 설정

```
# 상세 로그 활성화
RUST_LOG=debug  ./target/release/polkadot-parachain


# 특정 모듈 로그
RUST_LOG=cumulus_client_consensus_common=debug,cumulus_client_collator=debug
```

메트릭 수집

```
# Prometheus 메트릭 활성화
./target/release/polkadot-parachain  --prometheus-external
```

결론

Cumulus는 Substrate의 강력함을 파라체인 개발에 그대로 적용하면서, 릴레이 체인과의 통신, 크로스체인 메시징, 합의 메커니즘 등 파라체인 특화 기능을 제공하는 포괄적인 SDK입니다.

제공되는 파라체인 템플릿을 통해 개발자는 빠르게 파라체인 개발을 시작할 수 있으며, Omni Node와 Zombienet 같은 도구들을 활용하여 효율적인 개발 환경을 구축할 수 있습니다.

다음 섹션에서는 XCM(Cross-Consensus Messaging) 시스템에 대해 더 자세히 살펴보겠습니다.